

Technology Stack

Living inventory of technologies used in BankOne. **Update this document whenever dependencies, runtimes, or tooling change** (add, upgrade, remove, or replace). Source of truth for versions:

`BankOne-BackEnd/pom.xml` and `BankOne-Frontend/package.json`.

Last reviewed: 2026-08-02

At a glance

Layer	Primary stack
Backend Frontend Database	Java 21 · Spring Boot 4.1 · Maven · WAR Angular 22 · TypeScript 6 · npm · SCSS PostgreSQL (<code>bankone_write</code> ; optional <code>bankone_read</code> replica lab) Redis (local Docker; entity cache + Bucket4j) Apache Kafka (local Docker / Aiven on Render) Mailpit (local SMTP) · Twilio SendGrid HTTPS (Render) Spring Security · JWT (JJWT) Chart.js (UI) · OpenPDF (server PDF) <code>audit_event</code> activity trail + backfill Open Liberty (<code>:9080</code> / <code>:9443</code> , JDWP <code>:7777</code>) Render (Docker <code>-Pdocker</code>) · Netlify (Angular) Pandoc · Google Chrome (PDF) · python-docx TECH_LEARNING_PLAN.md

Backend (`BankOne-BackEnd/`)

Language & build

Technology	Version / notes	Where defined
Java	21	<code>pom.xml</code> <code>java.version</code> <code>.mvn/ pom.xml</code> <code>pom.xml</code> <code><packaging></code>

Spring & persistence

Technology	Purpose
<code>spring-boot-starter- webmvc</code>	Web / REST (<code>@RestController</code>)
<code>spring-boot-starter- data-jpa</code>	JPA repositories, entities, specifications
Hibernate (via Spring Data JPA)	ORM; <code>ddl-auto=update</code> ; SQL logging in <code>application.properties</code>
<code>spring-boot-starter- validation</code>	Bean Validation (<code>@Valid</code> , constraints)
<code>spring-boot-starter- actuator</code>	Actuator endpoints
<code>spring-boot-devtools</code>	Hot reload (runtime, optional)

Security & tokens

Technology	Version / notes
spring-boot-starter- security	Stateless session; filters; method/URL authorization
JJWT (jjwt-api, jjwt-impl, jjwt-jackson)	0.12.7 — HMAC JWT create/parse

Database & JDBC

Technology	Notes
PostgreSQL	Primary DB <code>bankone</code> ; optional read lab DB <code>bankone_read</code> (<code>app.replica.*</code>)
PostgreSQL JDBC driver	<code>org.postgresql:postgresql</code> (runtime)
schema.sql / data.sql	SQL init (<code>spring.sql.init.mode=always</code>)

Redis — cache-aside & rate limiting (local)

Technology	Purpose
Redis	Docker Compose service typically <code>:6379</code> (<code>REDIS_HOST</code> / <code>REDIS_PORT</code>)
spring-boot-starter- data-redis	Lettuce client; shared host for rate limit + entity cache
spring-boot-starter- cache	Spring <code>@Cacheable</code> / <code>@CacheEvict</code>
Bucket4j + Lettuce	Login / API rate-limit buckets when <code>app.rate-limit.redis-enabled=true</code>
Entity cache	<code>com.bankone.cache</code> — cache-aside for major reads; <code>app.cache.redis-enabled</code> (prod off); JDK value serialization; keys <code>bankone:<region>::...</code>
Docs	MODULES/Caching.md

Messaging & email

Technology	Purpose
spring-boot-starter- kafka	Kafka produce/consume (<code>com.bankone.notification</code>)
spring-boot-starter- mail	SMTP path (Mailpit locally)
Apache Kafka 3.9	Local: <code>docker compose service kafka</code> (<code>apache/kafka:3.9.0</code> :9092)
Aiven Kafka	Render managed broker (SASL_SSL + SCRAM + CA PEM)
Mailpit	Local fake inbox SMTP <code>:1025</code> / UI <code>:8025</code>
Twilio SendGrid	Render email via HTTPS API when <code>MAIL_TRANSPORT=sendgrid</code>

Code generation & container

Technology	Notes
Lombok	Optional; annotation processing in <code>maven-compiler-plugin</code>
spring-boot-starter- tomcat	Embedded Tomcat for <code>spring-boot:run</code> / local WAR (<code>provided</code> for Liberty deploy)

Test dependencies

Technology
spring-boot-starter-data-jpa-test spring-boot-starter-security-test spring-boot-starter-validation-test spring-boot-starter-webmvc-test

Frontend ([BankOne-Frontend/](#))

Core framework

Technology	Version / notes	Where defined
Angular	^22.0.x (core, router, forms, common, animations, compiler, platform-browser) ~6.0.2 ~7.8.0 ^2.3.0 11.16.0 (<code>packageManager</code>) Component / global styles	<code>package.json</code>

UI libraries

Technology	Version / notes
Angular Material	^22.0.5 — buttons, dialogs, forms, icons, cards, etc.
Angular CDK	^22.0.5 — Material peer dependency

Build & quality tooling

Technology	Version / notes
Angular CLI @angular/build Prettier Vitest jsdom	^22.0.7 ^22.0.7 — application builder ^3.8.1 ^4.0.8 — unit tests (<code>ng test</code>) ^28.0.0 — test DOM

Runtime patterns (no extra packages)

Concern	Approach
HTTP Auth	Angular <code>HttpClient</code> JWT in browser storage; Bearer interceptor; <code>authGuard</code> + roles App shell + sidebar under <code>core/layout</code>

Infrastructure & deployment

Technology	Role
Open Liberty	Primary app server (HTTP 9080, HTTPS 9443); <code>scripts/redeploy-liberty.sh</code>
Embedded Tomcat	Alternate local run via Spring Boot (<code>server.port=8080</code>)
PostgreSQL	Persistence (localhost:5432 typical); <code>bankone</code> (+ optional <code>bankone_read</code>)
Redis	Local cache + rate-limit store (<code>:6379</code>)

Documentation toolchain

Technology	Role
Markdown Pandoc Google Chrome python-docx	Working copies in <code>docs/_source/</code> Markdown → DOCX / HTML Headless print → PDF Post-process DOCX styling (<code>scripts/_polish_docx.py</code>) Rebuild all DOCX + PDF pairs

IDE / agent conventions (project)

Item	Notes
Cursor rules	<code>.cursor/rules/</code> (docs architect, teach-while-building, deferred hardening) Docs updated with every code change (see <code>documentation-architect.mdc</code>)

How to keep this file current

1. After changing `pom.xml` or `package.json` , update the matching tables above (name + version + purpose).
2. After changing deploy target, DB, or auth mechanism, update **At a glance** and the relevant section.
3. Append an entry to `CHANGELOG.md` .
4. Run `BankOne/scripts/regenerate-docs.sh` so DOCX/PDF stay aligned.

Do **not** list planned libraries here until they are actually added to the build.